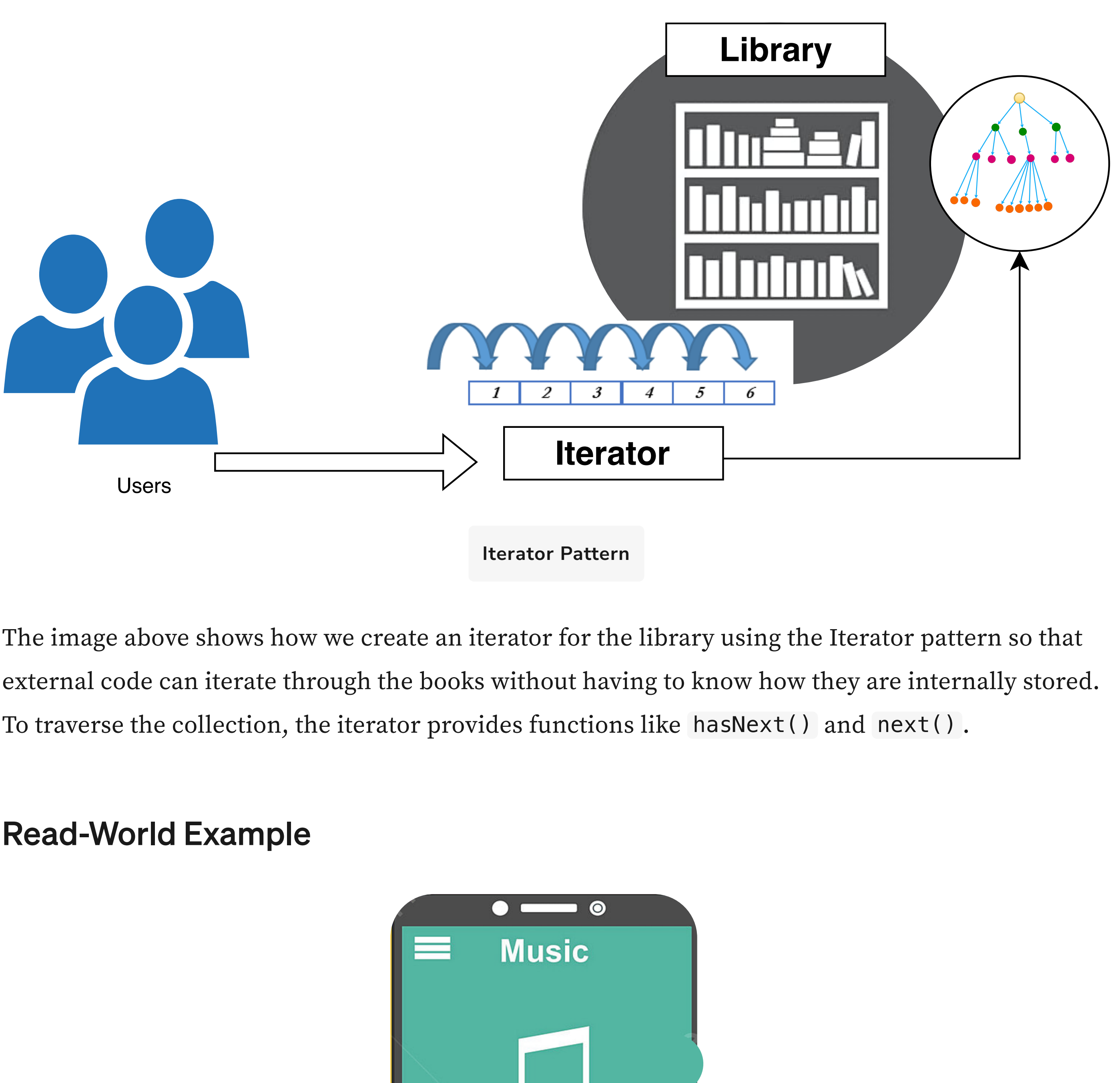
Iterator Pattern

Iterator Pattern provides a way to access elements of a collection object sequentially without exposing its underlying structure. This pattern introduces the concept of iterator. iterator is a tool through which you can traverse a collection (arrays, lists, or even more complex data structures) in a standardized manner.

Consider a scenario when you are working with a complex data structure, such as a graph or a tree. Extracting the data from such data structure is tricky, especially if don't want to show the internal structure of the data structure to the client. Without the use of an iterator pattern, the code to navigate through the elements will be complex and tightly coupled.

Suppose you have a library of books and want to display the titles of all the books. If you don't have an iterator pattern, you will be required to know how books are stored in the library. This will lead to an inefficient code that will be difficult to maintain.

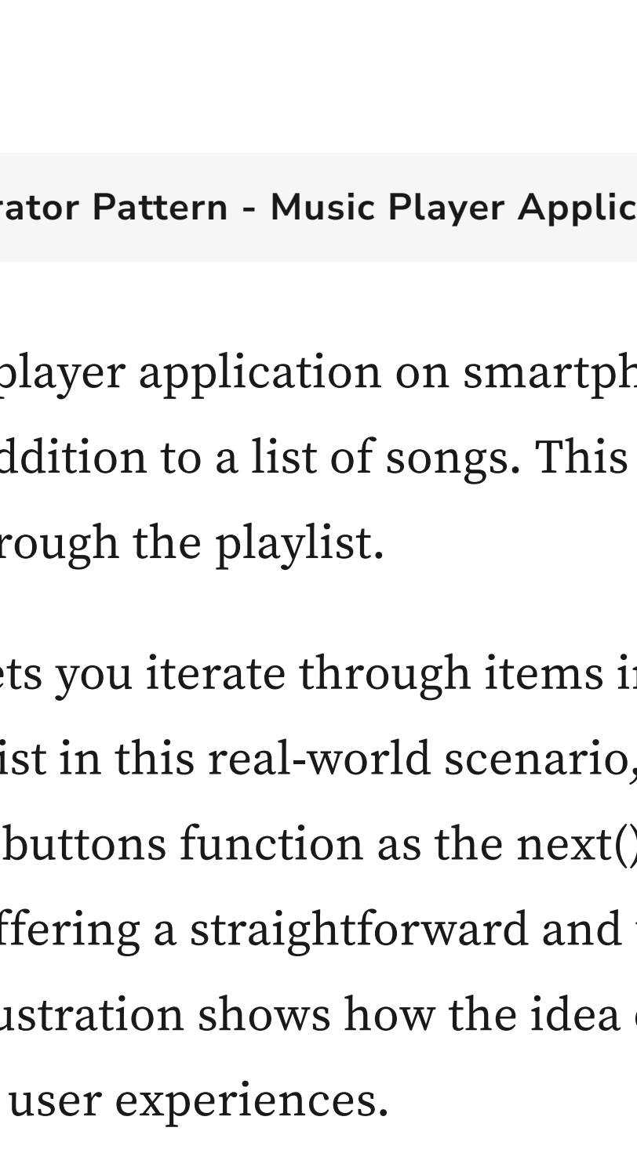
This is elegantly solved by the Iterator pattern, which separates the traversal mechanism from the data structure itself. It involves two key players: the 'Iterable' that provides the iterator and the 'Iterator' that encapsulates the traversal logic.



Iterator Pattern

The image above shows how we create an iterator for the library using the Iterator pattern so that external code can iterate through the books without having to know how they are internally stored. To traverse the collection, the iterator provides functions like `hasNext()` and `next()`.

Read-World Example

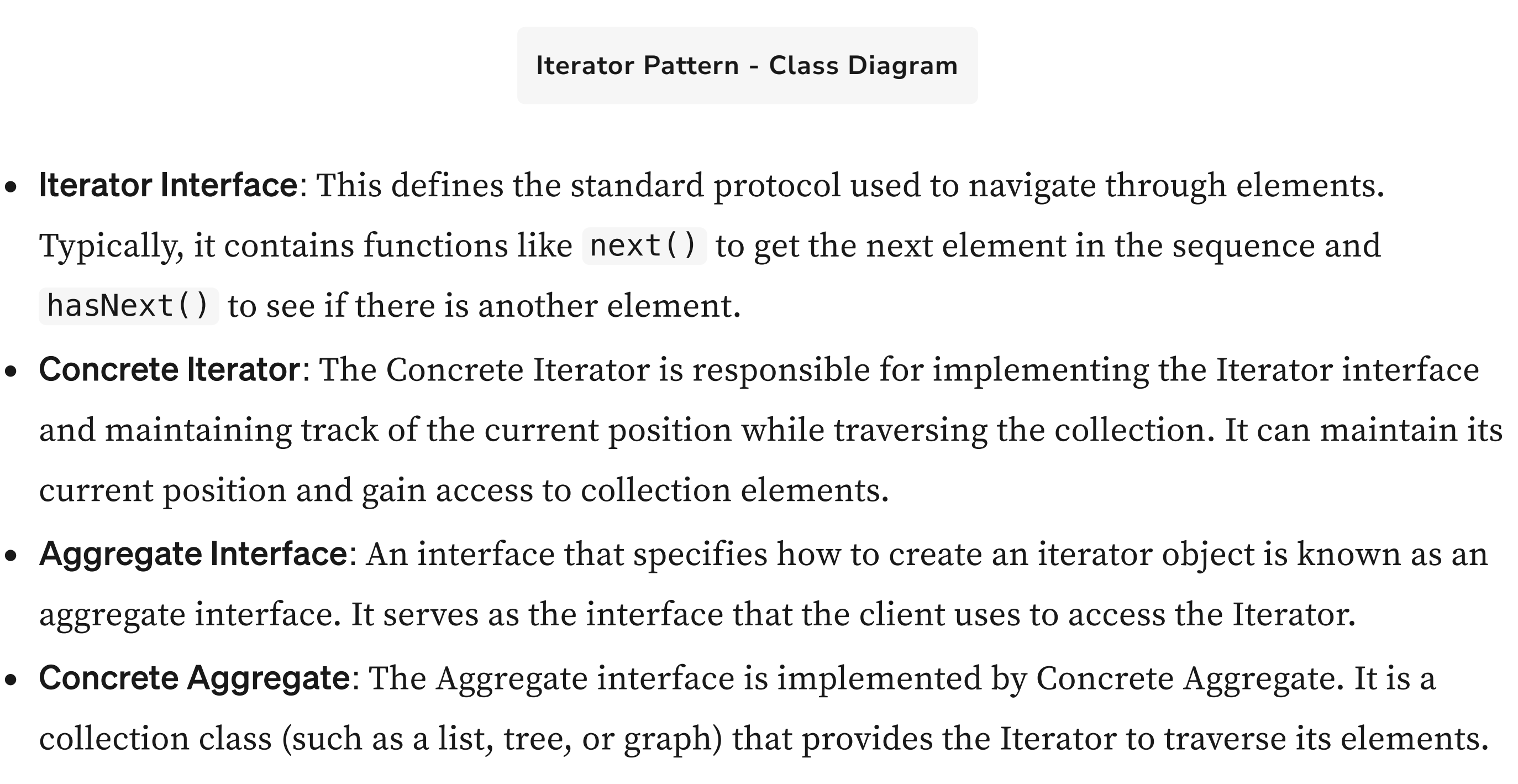


Iterator Pattern - Music Player Application

In the image, you can see a music player application on smartphone. There are 'next' and 'previous' buttons on the phone's screen in addition to a list of songs. This arrangement represents how the Iterator pattern is used to move through the playlist.

Like an iterator in programming lets you iterate through items in a collection, users can easily navigate through their music playlist in this real-world scenario, mirroring the Iterator pattern. The application's "next" and "previous" buttons function as the next() and previous() methods of an iterator's real-world equivalents, offering a straightforward and user-friendly method of accessing songs in the desired order. This illustration shows how the idea of iterators is present in both software development and regular user experiences.

Structure of Iterator Pattern



Iterator Pattern - Class Diagram

- **Iterator Interface**: This defines the standard protocol used to navigate through elements. Typically, it contains functions like `next()` to get the next element in the sequence and `hasNext()` to see if there is another element.
- **Concrete Iterator**: The Concrete Iterator is responsible for implementing the Iterator interface and maintaining track of the current position while traversing the collection. It can maintain its current position and gain access to collection elements.
- **Aggregate Interface**: An interface that specifies how to create an iterator object is known as an aggregate interface. It serves as the interface that the client uses to access the Iterator.
- **Concrete Aggregate**: The Aggregate interface is implemented by Concrete Aggregate. It is a collection class (such as a list, tree, or graph) that provides the Iterator to traverse its elements. It creates and returns an instance of the Concrete Iterator.

Implementation of Iterator Pattern

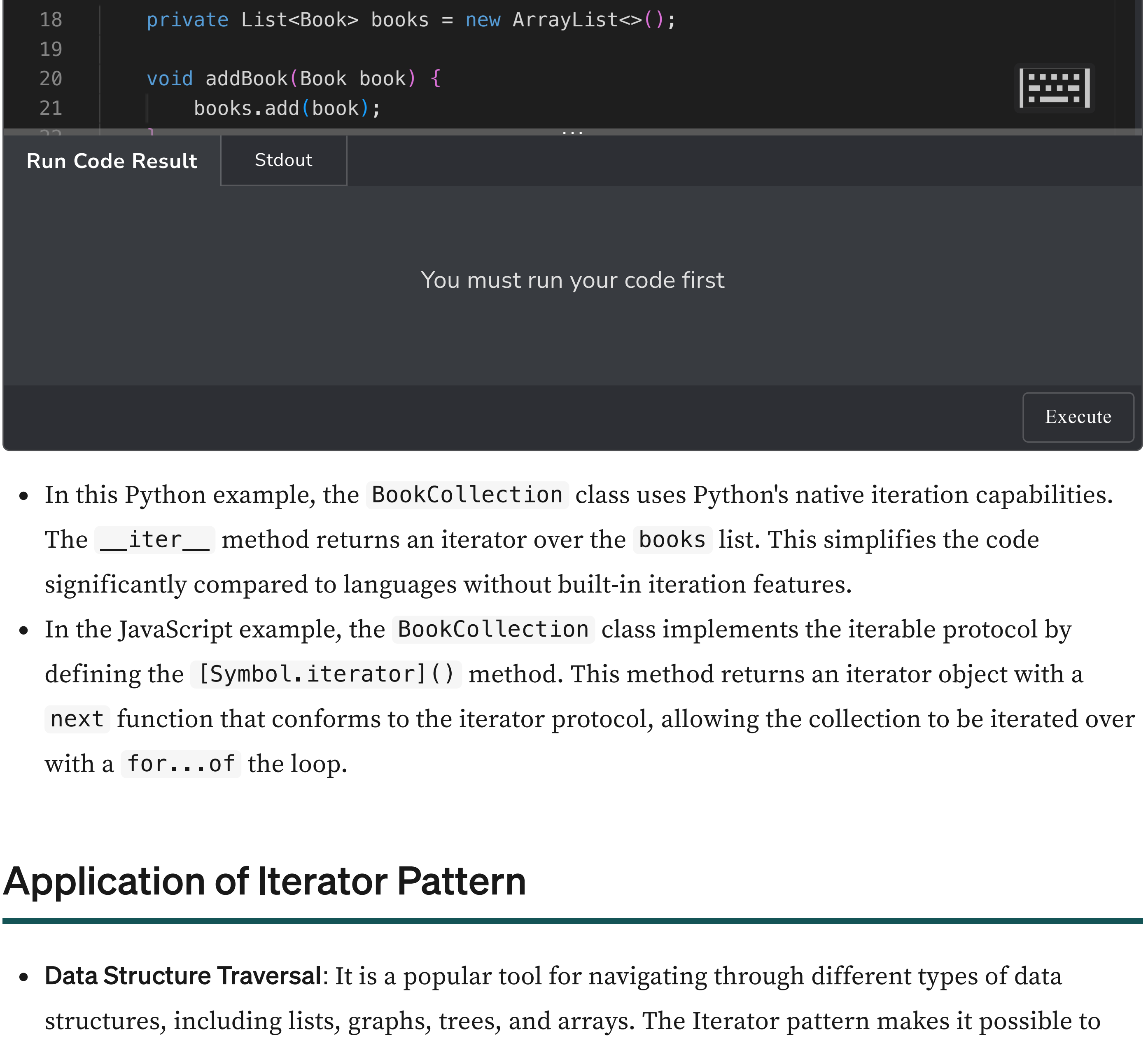
Let's consider a book collection in a library. We want to provide a way to iterate through the books without exposing the underlying data structure of the collection.

```
// Iterator Interface
interface Iterator {
    hasNext(): Boolean
    next(): Object
}

// Concrete Iterator for Book Collection
class BookIterator implements Iterator {
    private currentIndex = 0
    private collection: BookCollection

    BookIterator(BookCollection collection) {
        this.collection = collection
    }

    hasNext(): Boolean {
        return currentIndex < collection.getLength()
    }

    next(): Object {
        if (this.hasNext()) {
            book = collection.getAt(currentIndex)
            currentIndex += 1
            return book
        }
        return null
    }
}

// Aggregate Interface
interface Aggregate {
    createIterator(): Iterator
}

// Concrete Aggregate for Books
class BookCollection implements Aggregate {
    private books: Array

    BookCollection() {
        books = new Array()
    }

    addBook(book: Book) {
        books.append(book)
    }

    getLength(): Integer {
        return books.length
    }

    getAt(index: Integer): Book {
        return books[index]
    }

    createIterator(): Iterator {
        return new BookIterator(this)
    }
}

// Client Code
main() {
    bookCollection = new BookCollection()
    bookCollection.addBook(new Book("Book 1"))
    bookCollection.addBook(new Book("Book 2"))
    // Add more books ...

    iterator = bookCollection.createIterator()

    while (iterator.hasNext()) {
        book = iterator.next()
        print(book.title)
    }
}
```

- **Iterator Interface**: The iterator interface defines the `hasNext()` and `next()` methods. It provides a method of sequentially accessing elements within a collection without disclosing the collection's underlying structure.
- **BookIterator** (**Concrete Iterator**): Implements the Iterator interface for the book collection. It allows for sequential access to its elements and maintains track of the current position within the collection.
- **Aggregate Interface**: Provides a method for generating an iterator called `createIterator()`.
- **Concrete Aggregate** (**BookCollection**): Implements the Aggregate interface. It is a collection of books that has the ability to generate a `BookIterator` for iterating through them.
- **Client**: The client (similar to a library system) interacts with the collection via the Iterator. It accesses each book sequentially by requesting an Iterator from the BookCollection.

Let's look at the implementation of this example in multiple programming languages.

Implementation



```java
import java.util.ArrayList;
import java.util.List;

class Book {
    String title;

    Book(String title) {
        this.title = title;
    }
}

interface Iterator {
    boolean hasNext();
    Book next();
}

class BookCollection {
    private List<Book> books = new ArrayList<>();

    void addBook(Book book) {
        books.add(book);
```

You must run your code first

- In this Python example, the `BookCollection` class uses Python's native iteration capabilities. The `__iter__` method returns an iterator over the `books` list. This simplifies the code significantly compared to languages without built-in iteration features.
- In the JavaScript example, the `BookCollection` class implements the iterable protocol by defining the `[Symbol.iterator]()` method. This method returns an iterator object with a `next` function that conforms to the iterator protocol, allowing the collection to be iterated over with a `for...of` the loop.

Application of Iterator Pattern

- **Data Structure Traversal**: It is a popular tool for navigating through different types of data structures, including lists, graphs, trees, and arrays. The Iterator pattern makes it possible to access elements in these structures without revealing their internal details.
- **GUI Components**: Iterators are used in graphical user interfaces to traverse through elements like menus, grids, and lists, providing consistent and easy access to GUI components.
- **Aggregation of Objects**: Iterators are used in object-oriented programming to access the components of aggregate objects. For example, iterating through a group of objects sharing an interface.
- **Database Operations**: Iterators are used to iterate over records retrieved from databases. They provide a smooth method of navigating through search results.
- **File Processing**: Iterators can be used to read data from files in chunks or line by line without having to load the entire file into memory.
- **Network Communication**: Iterators can be used to iterate over a set of network nodes or connections in network applications.
- **Social Media Feeds**: Using iterators to scroll through feeds in social media apps, where the items in the feed can load dynamically as the user moves the cursor.
- **Game Development**: Collections of game objects or elements on a game board can be iterated over using iterators in game programming.

Pros and Cons

Pros	Cons
Separation of Concerns: Separates the collection's internal structure and the algorithm to traverse it.	**Complexity:** Can add extra complexity and overhead to simple collections.
Flexibility: Provides multiple ways to traverse a collection (forward, backward, filtered).	**Iterator Invalidation:** Iterators can become invalid if the collection is modified during iteration.
Abstraction and Consistency: Provides a uniform interface for multiple types of collections.	**Concurrent Modification:** Standard iterators typically don't support safe concurrent modification and iteration.
Control over Iteration: Gives more control over the pace and order of traversal.	**Statefulness:** Iterators maintain state, which needs careful handling to avoid errors.
Memory Efficiency: Efficient in memory usage, especially for large collections.	**Simplicity vs. Overhead:** For simple tasks, using an iterator might be more complex than necessary.

The Iterator pattern is a fundamental design pattern that is especially useful in scenarios that require a standardized way to access collection elements. Though it adds more levels of abstraction, making the codebase more structured and manageable, it's vital to take into account the possible overheads and complications it may cause.

← Previous
Command Pattern
Next →
Mediator Pattern
☐ Mark as Completed